ONLINE BOOK STORE MANAGEMENT SYSTEM

SQL DATABASE PROJECT REPORT

---

1. INTRODUCTION

1.1 Project Overview

The **Online Book Store Management System** is a relational database project developed using SQL. The main purpose of this project is to manage the different activities of an online bookstore, such as storing information about books, authors, publishers, customers, orders, and order details.

In a traditional bookstore, maintaining records manually can be difficult and time-consuming. A database management system provides an organized and efficient way to store, retrieve, update, and analyze information. This project demonstrates how SQL can be used to design and manage a bookstore database.

The database contains six main tables:

1. **Authors**
2. **Publishers**
3. **Books**
4. **Customers**
5. **Orders**
6. **OrderDetails**

These tables are connected using primary keys and foreign keys. The relationships between the tables allow the system to retrieve useful information, such as the books written by an author, the publisher of a book, the orders placed by customers, and the total revenue generated from delivered orders.

The project also contains 16 SQL practice queries covering important concepts such as:

- SELECT
- WHERE
- JOIN
- LEFT JOIN

- GROUP BY
- HAVING
- Aggregate functions
- ORDER BY
- LIMIT
- OFFSET
- Date functions
- Calculated fields

1.2 Purpose of the Project

The main purpose of the project is to create a simple and efficient database system for managing an online bookstore.

The system helps in:

- Maintaining book information.
- Maintaining author and publisher information.
- Maintaining customer records.
- Recording customer orders.
- Storing individual items within an order.
- Calculating order item amounts.
- Finding total revenue.
- Identifying popular books.
- Identifying customers with multiple orders.
- Finding books that are out of stock.
- Generating useful reports using SQL queries.

---

2. OBJECTIVES OF THE PROJECT

The major objectives of the Online Book Store Management System are:

2.1 Store Book Information

The system stores important details about every book, including its title, author, publisher, genre, price, stock quantity, and publication year.

2.2 Manage Authors and Publishers

The database maintains separate records for authors and publishers. This avoids unnecessary repetition of author and publisher information in the Books table.

2.3 Manage Customers

Customer information such as name, email, city, and phone number is stored in the Customers table.

2.4 Manage Orders

The Orders table stores information about orders placed by customers, including the order date and order status.

2.5 Manage Order Items

The OrderDetails table records which books are included in each order, along with their quantity and unit price.

2.6 Generate Business Reports

SQL queries can be used to generate reports such as:

- Total number of books written by each author.
- Average price by genre.
- Total revenue.
- Best-selling books.
- Customers with multiple orders.
- Books that have never been ordered.
- Publisher with the maximum number of books.

2.7 Practice Relational Database Concepts

This project demonstrates practical use of relational database concepts including:

- Primary keys

- Foreign keys
- Relationships
- Inner joins
- Left joins
- Aggregate functions
- Grouping
- Filtering
- Sorting

---

3. DATABASE DESIGN

The database consists of six tables.

3.1 Authors Table

The **Authors** table stores information about the authors of books.

Column	Data Type	Description
author_id	INTEGER	Primary key
author_name	VARCHAR(100)	Name of author
country	VARCHAR(50)	Author's country

The author_id uniquely identifies every author.

Sample authors include:

- R.K. Narayan
- J.K. Rowling
- Chetan Bhagat
- Agatha Christie
- Paulo Coelho
- Dan Brown

- Ruskin Bond

3.2 Publishers Table

The **Publishers** table stores publisher information.

Column	Data Type	Description
publisher_id	INTEGER	Primary key
publisher_name	VARCHAR(100)	Publisher name
city	VARCHAR(50)	Publisher's city

Sample publishers include Penguin Books, HarperCollins, Bloomsbury, Rupa Publications, and Random House.

3.3 Books Table

The **Books** table is one of the main tables in the system.

Column	Data Type	Description
book_id	INTEGER	Primary key
title	VARCHAR(150)	Book title
author_id	INTEGER	Foreign key
publisher_id	INTEGER	Foreign key
genre	VARCHAR(50)	Book genre
price	DECIMAL(10,2)	Book price
stock_quantity	INTEGER	Available stock
published_year	INTEGER	Year of publication

The author_id references the Authors table, while publisher_id references the Publishers table.

The database contains 12 books, including:

- Malgudi Days
- Harry Potter and the Sorcerer's Stone
- Half Girlfriend
- Murder on the Orient Express
- The Alchemist
- The Da Vinci Code
- The Blue Umbrella
- Five Point Someone
- And Then There Were None
- Harry Potter and the Chamber of Secrets
- Inferno
- Veronika Decides to Die

3.4 Customers Table

The **Customers** table stores information about customers.

Column	Data Type	Description
customer_id	INTEGER	Primary key
customer_name	VARCHAR(100)	Customer name
email	VARCHAR(100)	Email address
city	VARCHAR(50)	Customer city
phone	VARCHAR(15)	Phone number

The database contains six customers.

3.5 Orders Table

The **Orders** table records orders placed by customers.

Column	Data Type	Description
order_id	INTEGER	Primary key
customer_id	INTEGER	Foreign key
order_date	DATE	Date of order
status	VARCHAR(20)	Order status

The order status can contain values such as:

- Delivered
- Pending
- Cancelled

The customer_id connects every order with a customer.

3.6 OrderDetails Table

The **OrderDetails** table contains information about individual books included in an order.

Column	Data Type	Description
order_detail_id	INTEGER	Primary key
order_id	INTEGER	Foreign key
book_id	INTEGER	Foreign key
quantity	INTEGER	Number of copies
unit_price	DECIMAL(10,2)	Price per copy

This table creates the relationship between Orders and Books.

The total amount of an order item can be calculated as:

Total Amount = Quantity × Unit Price

---

4. DATABASE RELATIONSHIPS

The database follows a relational structure.

Author → Books

One author can write multiple books.

Authors (1) → (Many) Books

Publisher → Books

One publisher can publish multiple books.

Publishers (1) → (Many) Books

Customer → Orders

One customer can place multiple orders.

Customers (1) → (Many) Orders

Orders → OrderDetails

One order can contain multiple order items.

Orders (1) → (Many) OrderDetails

Books → OrderDetails

One book can appear in multiple order items.

Books (1) → (Many) OrderDetails

Therefore, Books and Orders have a many-to-many relationship through the OrderDetails table.

Relationship Structure

Authors

|

| 1 : Many

v

Books <--------- Publishers

|

| 1 : Many

v

OrderDetails

^

|

| Many : 1

Orders

^

|

| Many : 1

Customers

The use of foreign keys maintains referential integrity between these tables.

---

5. SQL QUERIES AND FUNCTIONALITY

The project contains 16 practice questions designed to demonstrate different SQL concepts.

Q1. Books with Author and Publisher

The first query uses JOIN to display the book title together with its author and publisher.

```
SELECT b.title, a.author_name, p.publisher_name
FROM Books b
JOIN Authors a ON b.author_id = a.author_id
JOIN Publishers p ON b.publisher_id = p.publisher_id;
```

This demonstrates how information from three related tables can be combined.

Q2. Books Priced Above 300

```
SELECT title, price
FROM Books
```

WHERE price > 300;

The WHERE clause filters books whose price is greater than 300.

This query demonstrates conditional filtering.

Q3. Out-of-Stock Books

SELECT title, stock_quantity

FROM Books

WHERE stock_quantity = 0;

This query identifies books that are currently unavailable.

According to the sample data, books such as **Harry Potter and the Sorcerer's Stone**, **The Blue Umbrella**, and **Veronika Decides to Die** have zero stock.

Q4. Number of Books by Each Author

SELECT a.author_name, COUNT(b.book_id) AS total_books

FROM Authors a

JOIN Books b ON a.author_id = b.author_id

GROUP BY a.author_name;

The COUNT() function counts books, while GROUP BY creates a separate result for each author.

Q5. Average Price by Genre

SELECT genre, ROUND(AVG(price),2) AS avg_price

FROM Books

GROUP BY genre;

The AVG() function calculates the average price of books within each genre.

Q6. Customers with More Than One Order

```
SELECT c.customer_name, COUNT(o.order_id) AS total_orders
FROM Customers c
JOIN Orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_name
HAVING COUNT(o.order_id) > 1;
```

The HAVING clause filters grouped results.

This query identifies customers who have placed more than one order.

Q7. Total Revenue from Delivered Orders

```
SELECT ROUND(SUM(od.quantity * od.unit_price),2) AS total_revenue
FROM OrderDetails od
JOIN Orders o ON od.order_id = o.order_id
WHERE o.status = 'Delivered';
```

The query calculates revenue using:

Quantity × Unit Price

Only orders having the status **Delivered** are included.

Q8. Top Three Best-Selling Books

```
SELECT b.title, SUM(od.quantity) AS total_sold
FROM OrderDetails od
JOIN Books b ON od.book_id = b.book_id
GROUP BY b.title
ORDER BY total_sold DESC
LIMIT 3;
```

The query uses SUM() to calculate the total quantity sold for each book.

ORDER BY ... DESC sorts the books from highest to lowest sales, and LIMIT 3 returns the top three.

Q9. Customers Who Never Placed an Order

```
SELECT c.customer_name
FROM Customers c
LEFT JOIN Orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

The LEFT JOIN ensures that customers without matching orders are included.

Q10. Books That Have Never Been Ordered

```
SELECT b.title
FROM Books b
LEFT JOIN OrderDetails od ON b.book_id = od.book_id
WHERE od.order_id IS NULL;
```

This query identifies books that do not appear in any order details.

Q11. Customer Who Spent the Most Money

```
SELECT c.customer_name,
    ROUND(SUM(od.quantity * od.unit_price),2) AS total_spent
FROM Customers c
JOIN Orders o ON c.customer_id = o.customer_id
JOIN OrderDetails od ON o.order_id = od.order_id
GROUP BY c.customer_name
ORDER BY total_spent DESC
LIMIT 1;
```

The query calculates total spending for each customer and returns the customer with the highest amount.

For an actual sales report, the query can additionally filter for delivered orders:

WHERE o.status = 'Delivered'

This prevents cancelled or pending orders from being treated as completed purchases.

Q12. Publisher with Maximum Books

SELECT p.publisher_name, COUNT(b.book_id) AS books_published

FROM Publishers p

JOIN Books b ON p.publisher_id = b.publisher_id

GROUP BY p.publisher_name

ORDER BY books_published DESC

LIMIT 1;

This query counts the number of books associated with every publisher and returns the publisher with the highest count.

Q13. Second Highest Priced Book

SELECT title, price

FROM Books

ORDER BY price DESC

LIMIT 1 OFFSET 1;

The books are sorted from highest price to lowest price.

OFFSET 1 skips the most expensive book, and LIMIT 1 returns the next one.

Q14. Books Published After 2000

SELECT title, published_year

FROM Books

```
WHERE published_year > 2000;
```

This query displays books whose publication year is greater than 2000.

Q15. Order Details with Customer and Book Information

```
SELECT c.customer_name,
    b.title,
    od.quantity,
    ROUND(od.quantity * od.unit_price,2) AS total_amount
FROM OrderDetails od
JOIN Orders o ON od.order_id = o.order_id
JOIN Customers c ON o.customer_id = c.customer_id
JOIN Books b ON od.book_id = b.book_id
ORDER BY o.order_id;
```

This query combines four tables to provide a useful order report containing:

- Customer name
- Book title
- Quantity
- Total amount

It demonstrates the practical use of multiple joins and calculated columns.

---

6. SAMPLE DATA SUMMARY

The database contains the following sample records:

Table	Number of Records
Authors	7
Publishers	5

Table	Number of Records
Books	12
Customers	6
Orders	9
OrderDetails	12

The sample data represents a small online bookstore containing books from different genres and authors.

The genres represented in the database include:

- Fiction
- Fantasy
- Romance
- Mystery
- Thriller

The orders contain three different statuses:

- Delivered
- Pending
- Cancelled

This makes the database suitable for demonstrating real-world reporting requirements.

---

7. TECHNOLOGIES USED

Database

SQL / SQLite

The queries use SQLite-compatible syntax, particularly the strftime() function used in Q15.

SQL Concepts Used

The project demonstrates:

- CREATE TABLE

- DROP TABLE
- INSERT INTO
- SELECT
- WHERE
- JOIN
- LEFT JOIN
- GROUP BY
- HAVING
- COUNT()
- SUM()
- AVG()
- ROUND()
- ORDER BY
- LIMIT
- OFFSET
- IS NULL
- strftime()
- Primary keys
- Foreign keys

---

8. ADVANTAGES OF THE SYSTEM

The Online Book Store Management System provides several advantages.

8.1 Organized Data

All bookstore information is stored in structured relational tables.

8.2 Reduced Data Duplication

Author and publisher information is stored separately and connected using foreign keys.

8.3 Easy Data Retrieval

SQL queries allow users to quickly find books, customers, orders, and sales information.

8.4 Better Reporting

The database can generate reports related to sales, customers, stock, publishers, and books.

8.5 Data Integrity

Primary keys uniquely identify records, while foreign keys establish valid relationships between tables.

8.6 Scalable Structure

Additional books, customers, orders, and authors can be added without changing the basic database structure.

---

9. LIMITATIONS

Although the project demonstrates the main concepts of an online bookstore database, it is a simplified academic project.

Some limitations are:

1. There is no login or authentication system.
2. Payment information is not stored.
3. Shipping information is not included.
4. Book reviews and ratings are not included.
5. There is no automated stock update after an order.
6. There is no discount or coupon management.
7. The database contains only sample data.
8. No graphical user interface is provided.
9. Order status is stored as simple text instead of using a separate status table.

These features could be added in a more advanced version of the project.

---

10. FUTURE ENHANCEMENTS

The system can be improved by adding the following features:

10.1 Payment Management

A separate Payments table can be created to store payment method, payment date, transaction status, and amount.

10.2 Shipping Management

A Shipping table can store delivery address, courier details, tracking number, and delivery status.

10.3 User Authentication

Customers can be provided with usernames and passwords to securely access their accounts.

10.4 Reviews and Ratings

Customers can give ratings and reviews for books they have purchased.

10.5 Automatic Stock Management

When a book is purchased, its stock quantity can automatically decrease.

10.6 Discounts

The system can support discount codes, promotional offers, and membership discounts.

10.7 Graphical User Interface

A web or desktop application can be connected to the database so that users can search books, place orders, and view order history.

---

11. CONCLUSION

The **Online Book Store Management System** successfully demonstrates the design and implementation of a relational database using SQL.

The project contains six related tables: Authors, Publishers, Books, Customers, Orders, and OrderDetails. Primary keys are used to uniquely identify records, while foreign keys establish relationships between related tables.

The sample data provides a realistic environment for practicing SQL queries. The 16 queries demonstrate important database operations such as joining tables, filtering records, grouping data, calculating averages and totals, sorting results, finding missing records, and generating sales reports.

The project also demonstrates how SQL can be used to answer practical business questions. For example, the database can identify out-of-stock books, find the best-selling books, determine customers with multiple orders, calculate delivered-order revenue, and identify publishers with the largest number of books.

Overall, this project provides a strong practical understanding of **relational database design and SQL query processing**. It can also serve as a foundation for developing a complete online bookstore application with a graphical interface, payment system, inventory management, and customer authentication in the future.

---

12. REFERENCES

1. SQL database concepts and relational database management system principles.
2. SQLite SQL syntax and built-in functions.
3. Classroom notes and SQL practice materials.
4. Practical implementation using CREATE TABLE, INSERT, SELECT, JOIN, GROUP BY, HAVING, and aggregate functions.

PROJECT SUMMARY

Project Title: Online Book Store Management System

Database: Online Book Store Database

Tables: 6

Sample Books: 12

Sample Authors: 7

Sample Publishers: 5

Sample Customers: 6

Sample Orders: 9

Practice Queries: 16

Main Technology: SQL / SQLite

Main Purpose: To manage books, authors, publishers, customers, orders, and order details and generate useful bookstore reports using SQL.